

Mangu Mail Phase 2 – Production-Grade Implementation

Overview: Phase 2 transforms the Mangu Mail application into a cloud-native, microservices-based architecture. We separate the frontend and backend into distinct repositories, implement a polished **React + Tailwind CSS** UI based on the provided `HTML_v1.html` design, and build out a suite of backend microservices. Key services include an **Email Sync Service** (for JMAP email synchronization via Kafka events), an **Email Processor Service** (handling SMTP send/receive with spam filtering and DKIM/SPF), an **AI Service** (FastAPI for email summarization, smart replies, and embeddings), an **Encryption Service** (using Signal Protocol with HashiCorp Vault for key management), and optional services like **Marketplace**, **Calendar**, and **Achievements**. We integrate **AWS Cognito** for all user authentication (replacing custom auth) and ensure each service verifies Cognito JWTs. Deployment is managed on **AWS** using Terraform-defined infrastructure: EKS (Kubernetes) for microservices, MSK (Managed Kafka) for the event bus, ElastiCache for Redis, Amazon S3 for storage, Amazon OpenSearch for indexing, and AWS Amplify for the frontend hosting. Infrastructure manifests (Kubernetes YAML or Helm charts) and GitOps configuration via ArgoCD are included. Each service is containerized with Docker and configured via environment variables (`.env.example` provided). The following sections detail the implementation of each component, with code excerpts and structure consistent with the Phase 1 scaffolding.

Project Structure

The project is divided into two main repositories (or directories): one for the frontend and one for all backend services (which can further be split per service). For clarity, we present a high-level structure as a monorepo with directories for each service:

```
mangu-mail-frontend/ # React + Tailwind CSS frontend (AWS Amplify hosting)
├── public/ # Static assets (index.html, favicon, etc.) ─── src/
│   └── assets/ # Images, icons, etc.
│       ├── components/ # UI Components (EmailList, EmailDetail, etc.) ─── features/ # Feature
│       │   ├── modules (email, ai, calendar, etc.) ─── layouts/ # Layout components (MainLayout, etc.) ───
│       │   └── pages/ # Page components (InboxPage, CalendarPage, etc.) ─── store/ # Redux store slices
│       │       ├── (emailSlice, uiSlice, etc.) ─── hooks/ # Custom React hooks
│       │       └── services/ # API service wrappers (if any)
│       └── index.js # App entry point
└── package.json # Frontend dependencies and scripts
```

mangu-mail-backend/ # Back-end services (could be split into multiple 1

```

repos)
├── email-sync-service/ # Service for JMAP email sync
│   ├── src/
│   │   ├── index.ts # Service entry point (JMAP client & Kafka producer)
│   │   ├── kafka/ # Kafka producer/consumer setup
│   │   └── ...
│   ├── Dockerfile
│   └── README.md
└── jmap/ # JMAP client logic

```

```

| └─ .env.example
└─ email-processor-service/ # Service for SMTP, spam filtering, DKIM/SPF | └─ src/
| | └─ server.ts # SMTP server setup and handlers
| | └─ dkim/ # DKIM signing utilities
| | └─ spam/ # SpamAssassin integration (e.g., spamc client) | └─ kafka/ # Kafka producer for events
(incoming mail, etc.) | | └─ ...
| └─ Dockerfile
| └─ README.md
| └─ .env.example
└─ ai-service/ # AI microservice (FastAPI Python) | └─ app/
| | └─ main.py # FastAPI app initialization
| | └─ routers/ # API endpoints (summarize, smart_replies, embed) | | └─ services/ # AI logic (calling
OpenAI or HuggingFace) | | └─ models/ # Pydantic models for requests/responses | | └─ ...
| └─ Dockerfile
| └─ README.md
| └─ .env.example
└─ encryption-service/ # Service for encryption & key management | └─ src/ (or app/ if Python)
| | └─ index.ts # Express app or FastAPI initialization | | └─ signal/ # Signal Protocol integration (using
libsignal) | | └─ vault/ # Vault client setup for key storage | | └─ routes/ # Endpoints (e.g., get public key,
encrypt, decrypt) | └─ Dockerfile
| └─ README.md
| └─ .env.example
└─ marketplace-service/ # (Optional) Service for email stakes marketplace | └─ ... (similar structure: src/,
Dockerfile, etc.)
└─ calendar-service/ # (Optional) Service for calendar events | └─ ...
└─ achievements-service/ # (Optional) Service for achievements tracking | └─ ...
└─ workflows-service/ # (Optional) Service for workflow automation 2

```

(Temporal)

```

| └─ src/
| | └─ worker.ts # Temporal worker setup
| | └─ workflows/ # Workflow definitions (TypeScript)
| | └─ activities/ # Activities (functions executed by workflows, e.g., sending emails)
| | └─ ...
| └─ Dockerfile
| └─ .env.example
└─ gateway-service/ # API Gateway (if used to aggregate routes) | └─ src/
| | └─ server.ts # Express server acting as API gateway/proxy | | └─ routes/ # Routes
forwarding to microservices or handling auth
| └─ ...
└─ package.json (if using a single repo with workspaces for Node services) └─ infra/ #
Infrastructure as code and deployment configs └─ terraform/ # Terraform scripts for AWS resources
| └─ main.tf, *.tf # EKS, MSK, S3, OpenSearch, Cognito, etc. └─ k8s-manifests/ # Kubernetes
YAML/Helm charts for services | └─ email-sync-deployment.yaml
| └─ email-processor-deployment.yaml
| └─ ... etc.

```

- | └─ argo-cd/ # ArgoCD application manifests
- └─ README.md # Instructions for provisioning and deploying

This structure ensures clear separation of concerns. The **frontend** is a standalone React app. The **backend** consists of multiple independent services, each with its own codebase, Docker image, and configuration. We include **README.md** files in each service directory explaining its purpose, setup, and environment variables. All sensitive configuration (API keys, database URLs, etc.) is provided via environment variables, with sample values in `.env.example` for ease of setup. Below we detail each major component of the system.

Frontend – React & Tailwind UI

The Mangu Mail frontend is implemented in **React** (using functional components and Hooks) with **Tailwind CSS** for styling, closely following the provided design in `HTML_v1.html`. We leverage modern React libraries for state management and routing:

- **React Router v6** for client-side routing (navigating between Inbox, Calendar, etc.).
- **Redux Toolkit** for global state management of emails, user profile, UI state, etc.
- **Tailwind CSS** for utility-first styling that matches the design (the Tailwind classes from the HTML design are used in JSX).

The UI is structured with a main layout and several feature pages: - A persistent sidebar navigation (with links for **Inbox**, **Aliases**, **Kanban** workflows, **Marketplace**, **Social** Graph/Feed, **Calendar**, **Achievements**,

3

etc., as seen in the HTML design). - A top header bar (with branding, user menu, etc.). - The main content area displays the selected view, e.g., Inbox list, Compose modal, Calendar, etc.

We created React components corresponding to the HTML elements: - Sidebar and Header components for layout. - EmailList component to render a scrollable list of emails, composed of EmailRow sub-components (each showing an email's subject, sender, etc.). - EmailDetail component for viewing a single email content (with replies, etc.). - ComposeModal component for composing a new email. This includes fields for recipients, subject, body, and integrates AI features (like **Smart Compose** suggestions and **sentiment/effectiveness** analysis). - AIAssistModal for showing AI-generated summaries or reply suggestions on-demand. - UI widgets like SmartReplies (buttons for suggested replies), AIAssistModal (with AI summary), a VoiceNote recorder component (allowing users to record and transcribe voice notes), MarketplaceWidget (showing email stake marketplace info, e.g., balance or offers), and an AchievementsPanel (displaying user achievements and points). - Utility components: FocusedTabs (e.g., toggling between "Focused" vs "Other" inbox, or different filters), WorkflowCards (for a Kanban board representation of email workflows), a floating action button for compose, and a Toast component for user notifications.

All components are styled via Tailwind classes (with a custom theme defined in global CSS or Tailwind config). Dark mode is supported using CSS variables toggled by a `data-theme="dark"` attribute (as seen in Phase 1 styles). For example, the MainLayout applies a background and text color based on the current theme.

State Management: The app's state uses Redux slices: - emailSlice manages the list of emails, fetching emails from the backend (`/api/emails`) and handling actions like send, reply, or snooze. - uiSlice manages UI state (sidebar open/closed, modals visibility, current active view, theme toggle, toast messages,

etc.). - userSlice holds user profile and preferences (name, email, dark mode setting, etc., as well as achievement stats like streaks and points). - Additional slices for each feature: aliasesSlice , calendarSlice , workflowsSlice , marketplaceSlice , socialSlice , achievementsSlice , and aiSlice . These slices have initial dummy data and reducers to update state, as well as async thunks for API calls where applicable. For example, aliasesSlice might define fetchAliases (GET / aliases from backend) and createAlias (POST new alias) actions via createAsyncThunk . The aiSlice defines thunks generateSummary and generateSmartReplies that call the AI backend endpoints and store results.

Below is a snippet illustrating how the **AI integration** works on the frontend, using Redux Toolkit to call the backend AI service and store the results:

```
// src/features/ai/aiSlice.js
import { createSlice, createAsyncThunk } from '@reduxjs/toolkit'; import axios from
'axios';

const API_BASE_URL = process.env.REACT_APP_API_BASE_URL || 'https://
api.mangumail.com'; // assume base URL of API Gateway

// Thunk to request an AI-generated summary for a given email
export const generateSummary = createAsyncThunk('ai/generateSummary', async (

(emailId) => {
  const response = await axios.post(`${API_BASE_URL}/ai/summarize`, { emailId });
  return response.data;
});

// Thunk to request smart reply suggestions
export const generateSmartReplies = createAsyncThunk('ai/generateSmartReplies', async (emailId) => {
  const response = await axios.post(`${API_BASE_URL}/ai/smart-replies`, { emailId });
  return response.data;
});

const aiSlice = createSlice({
  name: 'ai',
  initialState: { summaries: {}, smartReplies: {}, isProcessing: false, activeAction: null },
  reducers: {
    setActiveAction: (state, action) => {
      state.activeAction = action.payload; // e.g., 'summarizing' or 'suggesting_replies'
    }
  },
  extraReducers: (builder) => {
    builder
      .addCase(generateSummary.pending, (state) => {
        state.isProcessing = true;
      })
      .addCase(generateSummary.fulfilled, (state, action) => {
        const { emailId, summary } = action.payload;
        state.summaries[emailId] = summary;
      })
  })
});
```

```

        state.isProcessing = false;
    })
    .addCase(generateSmartReplies.fulfilled, (state, action) => {
        const { emailId, replies } = action.payload;
        state.smartReplies[emailId] = replies;
    });
}
});

export const { setActiveAction } = aiSlice.actions;
export default aiSlice.reducer;

```

The UI components (e.g., the Compose modal or an open email) dispatch these thunks to get AI results, and then display the returned summary or reply suggestions. For instance, when the user clicks "Summarize

5

Email", the app dispatches generateSummary(emailId) , and a loading state is shown until the AI service responds with the summary text, which is then rendered in the UI.

Tailwind Integration: We use a Tailwind configuration that extends the default theme with our custom colors (as defined in Phase 1 CSS variables). For example, we define primary and secondary color palettes in tailwind.config.js to match the gradients and shades used in the HTML design (indigo/purple gradients for icons, slate colors for text, etc.). The globals.css (imported in index.js) sets up base styles and CSS variables for light/dark mode, which Tailwind utility classes reference.

Frontend Authentication (AWS Cognito): The frontend does not implement its own login screen or store passwords. Instead, we integrate AWS Cognito using AWS Amplify or the Amazon Cognito JS SDK: - We created a Cognito User Pool for Mangu Mail users. The frontend uses Amplify's Auth module or Cognito hosted UI to handle user sign-up and sign-in. For example, using Amplify:

```

import { Auth } from 'aws-amplify';
// ... after user signs in:
const user = await Auth.signIn(username, password);
const jwtToken = user.signInUserSession.idToken.jwtToken;
// store jwtToken and use in API calls

```

We configure Amplify with the User Pool ID, App Client ID, and federation if needed. Alternatively, a simple approach is to use Cognito's hosted login page and redirect back with tokens.

- Once logged in, the app obtains a **JWT (ID token or Access token)** from Cognito. This token is stored (in memory or localStorage) and attached to all API requests to backend services as an Authorization: Bearer <JWT> header. The JWT contains the user's identity and is signed by Cognito.
- **Amplify for Hosting:** We use AWS Amplify Console to host the frontend. The repository is connected to Amplify for CI/CD deployment – Amplify will build the React app (running npm run build) and

deploy the static files to an S3 bucket with CloudFront CDN. (Alternatively, we could use S3 + CloudFront directly, but Amplify streamlines this.)

The end result is a responsive, single-page application that closely matches the design in HTML_v1.html – including interactive elements like voice note recording (using the MediaRecorder API with a VoiceNote component), drag-and-drop support for workflow Kanban (using a library like react-beautiful-dnd as added in dependencies), chart components for any analytics (e.g., using recharts for any visualizations), and toast notifications for feedback (using a library like react-hot-toast for nice notifications).

Backend Microservices

Phase 2's backend is composed of specialized microservices, each focusing on a specific domain of the application. They communicate via RESTful APIs, **Kafka** event streams, and, for long-lived processes, **Temporal.io** workflows. All services share common concerns: they are containerized, deployed to K8s, and

6

secured by AWS Cognito JWTs (meaning each incoming request must present a valid token from our User Pool). Below, we describe each service and provide implementation details:

1. Email Sync Service (JMAP Integration & Kafka)

The **Email Sync Service** is responsible for synchronizing emails between external mail stores and our application's database, using the **JMAP** protocol (JSON Meta Application Protocol). JMAP is a modern, ¹ efficient alternative to IMAP/SMTP for mail synchronization. Instead of the app polling via IMAP, this service can use JMAP's push or query capabilities to fetch updates in a single HTTP call, which is ideal for ¹ web/mobile clients.

Use Cases: If a user links an external email account (for example, their corporate email), the sync service would periodically fetch new messages via JMAP and publish them to our system. Or, if Mangu Mail itself acts as an email server, this service could expose JMAP endpoints or use JMAP internally for its mailbox storage.

Implementation: We implement the Email Sync service in TypeScript (Node.js) for consistency with the existing backend. It runs as a daemon service (no public HTTP API, or minimal API) performing these tasks: - Connect to a JMAP server (either an external provider's JMAP API or an internal mail store, e.g., Apache James which supports JMAP). - **Polling or Push:** Depending on capabilities, either poll for changes on a schedule or maintain an EventSource (JMAP can provide a push event stream of updates). - When new mail arrives or mail is updated/deleted, the service obtains the relevant data (messages, mailboxes, etc.) via JMAP API calls. For example, using an open-source JMAP client library (like jmapio/jmap-js in Node or making direct HTTP calls to JMAP endpoints). - The service then publishes events to **Apache Kafka** (AWS MSK). For instance, for a new email it might produce a message on a topic emails.new with the email details (sender, recipients, subject, body, etc.). This event can be consumed by other services (Email Processor, AI service for auto-summarization, etc.). - It may also write the new email to the primary data store (ScyllaDB) for permanent storage, and trigger indexing (via OpenSearch) for search capabilities. - If a user sends an email from Mangu Mail, the sync service might also handle updating the "Sent" folder or syncing that out to an external account via JMAP **EmailSubmission**.

Kafka Integration: The service uses a Kafka producer and consumer (e.g., kafkajs or Node RDKafka library). Topics might include: - email.sync.new – produced when a new email is fetched via JMAP. - email.sync.updated

- for read/unread or flag changes. - It could also consume a topic like email.send.request if the system routes outgoing email requests through Kafka (though typically sending might be direct via SMTP).

Code Snippet: Below is a simplified excerpt of the Email Sync Service's logic connecting to JMAP and producing Kafka events:

```
// email-sync-service/src/index.ts
import { JmapClient } from './jmap/client'; // Hypothetical JMAP client module import { Kafka, logLevel } from
'kafajs';
import dotenv from 'dotenv';

dotenv.config();

const JMAP_BASE_URL = process.env.JMAP_BASE_URL!; // e.g., JMAP server URL const
JMAP_ACCOUNT = process.env.JMAP_ACCOUNT_ID!; // Account ID or username
const JMAP_TOKEN = process.env.JMAP_TOKEN!; // Auth token for JMAP (could be from OAuth2 to
external service)

const KAFKA_BROKER = process.env.KAFKA_BROKER!; // e.g., MSK bootstrap server
const kafka = new Kafka({ clientId: 'email-sync-service', brokers: [KAFKA_BROKER],
logLevel: logLevel.INFO });
const producer = kafka.producer();

async function syncLoop() {
  const jmap = new JmapClient(JMAP_BASE_URL, JMAP_TOKEN);
  await producer.connect();
  console.log("Email Sync Service started. Polling for new emails...");

  // Example: periodic poll every 60 seconds
  setInterval(async () => {
    try {
      const newMessages = await jmap.getEmailsSinceLastCheck(JMAP_ACCOUNT); for (const
      msg of newMessages) {
        // Produce a Kafka event for each new email
        await producer.send({
          topic: 'emails.new',
          messages: [
            {
              key: msg.id,
              value: JSON.stringify({
                id: msg.id,
                threadId: msg.threadId,
                mailboxIds: msg.mailboxIds,
                from: msg.from,
                to: msg.to,
                subject: msg.subject,
                date: msg.receivedAt,
```

```

        snippet: msg.preview, // JMAP can provide a snippet
        flags: msg.keywords // unread/read flags
    })
  }
]
});
console.log(`Published new email event for ID: ${msg.id}`);
}
} catch (err) {
  console.error("Error during JMAP sync:", err);
}
}, 60000);

8
}

```

```

syncLoop().catch(err => console.error("Email Sync Service failed to start:", err));

```

In this snippet, JmapClient would handle the HTTP calls to the JMAP API (for example, using the JMAP / query and /get endpoints to find new messages since a certain state). It's simplified here; in practice, one might store a state token from the last sync to pass to JMAP for delta updates.

The **JMAP integration** allows efficient synchronization of mailboxes². Notably, JMAP can batch requests and send all data (headers, body, attachments metadata) in one round-trip, which is more efficient than¹ traditional IMAP. By using JMAP, the Email Sync Service ensures Mangu Mail stays up-to-date with the latest emails across user accounts.

2. Email Processor Service (SMTP, Spam Filtering, DKIM/SPF)

The **Email Processor Service** handles the lifecycle of sending and receiving emails within Mangu Mail's domain. It essentially functions as a mail transfer agent (MTA):

- Accepting incoming emails (via SMTP) for Mangu Mail users.
- Sending outgoing emails (relaying via SMTP or an API like AWS SES).
- Processing each email through security and classification steps: **Spam filtering** (SpamAssassin), **virus scanning** (if needed), and applying **DKIM signatures** and **SPF** checks.

We have two choices: integrate a full-fledged mail server like **Apache James** or implement these functions in our service code. Apache James is a robust Java mail server that supports SMTP, IMAP, and even JMAP, with built-in maillets for spam filtering and more. However, to maintain control and consistency with our Node/Python stack, we implement a custom lightweight SMTP server using Node.js.

Incoming Email (SMTP Server): We use a Node library such as smtp-server (which allows spinning up an SMTP listener in code). The Email Processor runs an SMTP server on a port (e.g., 25 or 2525 in the container; in AWS, we'd front it with an SMTP endpoint). When an incoming SMTP connection is made:

- We parse the incoming email data using a library like mailparser to get structured info (headers, body, attachments).
- We verify the recipient address to see if it's a valid user in our system (or an alias).
- We run the content through **SpamAssassin**. This can be done by calling out to a SpamAssassin daemon via its API or CLI (spamc). For example, the service could spawn a child process spamc -c to get a spam score for the raw email content, or use a Node binding if available. The result might tag the email as spam or give a score which we attach as

metadata. - We check **SPF** records if the sending domain is available (to detect spoofing). This involves doing a DNS lookup for SPF TXT records and verifying the connecting IP – there are libraries for SPF check, or we can delegate this to SpamAssassin as well (SpamAssassin will include SPF and DKIM checks if properly configured). - If the email passes checks (or even if spam, we might still accept but flag it), we then emit an event to Kafka (emails.received) and save the email to storage (ScyllaDB) under the user's inbox. We might also index it in OpenSearch for searchability. - If the email was encrypted (Mangu-to-Mangu user, see Encryption Service), we forward the encrypted payload to the Encryption Service to decrypt it before processing (or store it encrypted and only decrypt on-demand when user reads it – design choice). - Finally, we acknowledge the SMTP transaction (sending a 250 OK back to the sender).

9

We will also use DKIM verification for incoming emails: using a library like node-dkim to verify the DKIM Signature header against the sender's DNS public key, adding that info to the email's metadata (for trust indicators).

Outgoing Email: For sending emails composed by our users: - The front-end will call a backend API endpoint (e.g., POST /api/emails or /api/send) with the email content. We could handle this in an API Gateway or in this Email Processor service (exposing an HTTP endpoint for send). - The Email Processor service will receive the send request (likely via REST call or via a Kafka event from the API gateway), construct an email message (with MIME encoding of attachments, etc.), and send it out. We can use Node's

nodemailer library configured with an SMTP transport (e.g., pointing to an external SMTP relay like Amazon SES, or directly doing DNS MX resolution to send). - Before sending, we apply a **DKIM signature** to the email if it's from our domain. We'll have a DKIM private key stored (in Vault or an environment variable). Using a library (e.g., nodemailer has a DKIM option or using dkim-signer library), we sign the outgoing email headers so that recipients can verify our domain's authenticity. - Also, ensure the "Return Path" or envelope-from matches a domain with an SPF record that includes our sending IP (for SPF pass). - The service also logs/produces an event emails.sent on Kafka so other services (like Achievements or Social features) can know an email was sent (for example, to increment a "quick reply" streak if applicable). - It saves the sent email in the user's Sent mailbox in ScyllaDB.

DKIM/SPF Setup: We have published DNS records for our domain (e.g., mangumail.com) with a DKIM public key (TXT record) and an SPF record (listing the servers or include SES if using SES). The service's config (.env) will contain the DKIM private key and selector. For example, .env.example for this service includes:

```
SMTP_PORT=2525
DKIM_PRIVATE_KEY="-----BEGIN RSA PRIVATE KEY-----\nMIIIEogIBAAKCAQEA7...\n----- END RSA
PRIVATE KEY-----"
DKIM_KEY_SELECTOR="mangu"
DKIM_DOMAIN="mangumail.com"
SPAMASSASSIN_HOST="spamassassin" # if running spamd as a sidecar or service
SPAMASSASSIN_PORT=783
```

Code Snippet: The following illustrates how the Email Processor sets up an SMTP server and processes incoming mail:

```
// email-processor-service/src/server.ts
import { SMTPServer } from 'smtp-server';
import MailParser from 'mailparser'; // could use simpleParser from 'mailparser'
import { verifyDKIM } from './dkim/verify';
import { checkSPF } from './spam/spf';
import { spawn } from 'child_process';
import { Kafka } from 'kafkajs';
import { saveEmailToDB } from './storage';
```

```

const kafka = new Kafka({ clientId: 'email-processor', brokers:
10
[process.env.KAFKA_BROKER!] });
const producer = kafka.producer();
await producer.connect();

const smtpServer = new SMTPServer({
  authOptional: true, // we accept without SMTP auth (since it's incoming from other MTAs)
  onData(stream, session, callback) {
    MailParser.simpleParser(stream, async (err, parsed) => {
      if (err) {
        console.error("Error parsing email:", err);
        return callback(err);
      }
      try {
        const { from, to, subject, text, html, attachments, headers } = parsed; console.log(`Received
email from ${from?.text} to ${to?.text}: "$ {subject}"`);
        // SpamAssassin check (spawn spamc -c to get score)
        let spamScore = 0;
        try {
          const spamc = spawn('spamc', ['-c']); // expects spamd running
          spamc.stdin.write(parsed.raw);
          spamc.stdin.end();
          spamc.stdout.on('data', data => {
            const output = data.toString();
            // spamc -c output format: "<score> <threshold>\n"
            spamScore = parseFloat(output.split(' ')[0]);
          });
        } catch (e) {
          console.error("SpamAssassin check failed:", e);
        }
        // DKIM verification
        const dkimPass = await verifyDKIM(headers);
        // SPF check
        const spfPass = await checkSPF(session.remoteAddress,
parsed.from?.value[0].address);

        // Construct email object to save and send to Kafka

```

```

const emailRecord = {
  id: generateId(), // function to generate internal ID
  from: parsed.from.value,
  to: parsed.to.value,
  subject,
  text,
  html,
  attachments: attachments.map(a => ({ filename: a.filename,
contentType: a.contentType })),

```

```

                                                                    11
    spamScore,
    dkimVerified: dkimPass,
    spfVerified: spfPass,
    receivedAt: new Date().toISOString(),
    flags: { unread: true }
  };
  // Save to ScyllaDB (Cassandra)
  await saveEmailToDB(emailRecord);
  // Produce event to Kafka
  await producer.send({
    topic: 'emails.received',
    messages: [{ key: emailRecord.id, value:
JSON.stringify(emailRecord) }]
  });
  console.log(`Email ${emailRecord.id} processed and saved.`);
  callback(); // acknowledge SMTP reception
} catch (e) {
  console.error("Error processing email:", e);
  callback(e);
}
});
}
});
smtpServer.listen(process.env.SMTP_PORT || 2525);

```

In this code, when an email is received, we parse it, run spam checks, verify DKIM/SPF, then store the email and send a Kafka message. The actual implementation would need proper error handling and possibly using asynchronous SpamAssassin libraries, but this shows the flow.

For **sending emails**, we could similarly use NodeMailer:

```

// email-processor-service/src/sendMail.ts
import nodemailer from 'nodemailer';
import { Signer } from 'dkim-signer';

const transporter = nodemailer.createTransport({
  host: 'email-smtp.us-east-1.amazonaws.com', // if using SES SMTP, for example port: 587,

```

```

    auth: { user: process.env.SMTP_USER, pass: process.env.SMTP_PASS } });

// Function to send email with DKIM signing
export async function sendEmail(message: { from: string, to: string, subject: string, text: string, html?:
string }) {
    // DKIM signing options
    const dkimOptions = {

        privateKey: process.env.DKIM_PRIVATE_KEY!,
        keySelector: process.env.DKIM_KEY_SELECTOR!,
        domainName: process.env.DKIM_DOMAIN!
    };
    // nodemailer supports DKIM signing directly:
    transporter.sendMail({ ...message, dkim: dkimOptions }, (err, info) => { if (err) {
        console.error("Failed to send email:", err);
    } else {
        console.log("Email sent:", info.messageId);
        // Optionally produce event 'emails.sent' to Kafka here
    }
    });
}

```

This service would expose an HTTP endpoint (protected by Cognito JWT) for the frontend to call when sending an email, which internally calls `sendEmail`. Alternatively, the frontend call could go to a separate API gateway service that puts a message on a Kafka topic for outgoing email, which this service consumes and then sends – but for simplicity, a direct REST call to Email Processor is fine, as long as the JWT is verified.

3. AI Service (FastAPI for NLP Tasks)

The **AI Service** provides intelligent features to enhance the email experience. It's implemented in **Python** (FastAPI framework) to utilize powerful NLP libraries and tools. The service encapsulates all AI/ML tasks, such as:

- **Summarizing Emails:** Given an email (or thread) content, produce a concise summary.
- **Smart Reply Suggestions:** Given an email content, generate a few suggested replies (as short sentences the user can one-click insert).
- **Embedding Generation:** Convert email text to a vector embedding for semantic search or classification (e.g., for finding related emails or clustering).
- **Sentiment Analysis & Tone:** (From the UI design, there's a "sentiment and effectiveness" meter – the AI service can analyze the tone of an email or its likely effectiveness and return scores or feedback.)
- Possibly other features like **translation** or **text morphing** (Phase1 mentioned an `/api/ai/morph` endpoint).

Tech stack: FastAPI makes it easy to expose REST endpoints in Python. For the AI models:

- We can use OpenAI's API (via `openai` Python package) for things like summarization and reply suggestion to leverage GPT-4 or other models, since this yields high-quality results quickly.
- Alternatively, use local models from Hugging Face Transformers for summarization (e.g., PEGASUS or T5) and text generation (GPT-2/3 like models for replies). But given production-grade requirement, OpenAI might be preferred for accuracy (with the API key provided in config).
- For embeddings, OpenAI offers text embeddings (like `text-embedding-ada-002`), or we could use a smaller model like Sentence Transformers (sentence-transformers library) to generate vectors.
- Sentiment can be done via a simple model (e.g., TextBlob or a Hugging Face sentiment

model), and effectiveness might be a custom heuristic or ML model (perhaps analyzing length, politeness, reading level, etc.).

Endpoints: The service defines endpoints such as: - `POST /summarize` – request body contains emailId or actual text; the service fetches the email text (either via a direct DB query or the text could be sent in the request) and returns a summary string. - `POST /smart-replies` – request body with emailId or text; returns a list of reply suggestions (strings). - `POST /embed` – request with text or

13

emailId; returns an embedding vector (and possibly stores it in a vector index). - (Optional) `POST /analyze-tone` or similar – returns sentiment score or tone classification (could combine with summarization response).

We ensure that this service is protected (each request should include a JWT which we verify, because it's user-specific data we're processing). Since FastAPI can integrate authentication dependencies, we might implement a dependency that checks the Cognito JWT (see **Authentication** section below).

Code Snippet: Here's a simplified FastAPI setup for the AI Service:

```
# ai-service/app/main.py
from fastapi import FastAPI, Depends, HTTPException
from pydantic import BaseModel
import openai
import os
from .auth import verify_token # a dependency to verify Cognito JWT
openai.api_key = os.getenv("OPENAI_API_KEY")

app = FastAPI(title="ManguMail AI Service")

class EmailText(BaseModel):
    emailId: str = None
    text: str = None

@app.post("/summarize")
async def summarize_email(payload: EmailText, token: dict = Depends(verify_token)):
    """Summarize the email content."""
    # Retrieve email text: either from payload or fetch from DB by emailId
    text = payload.text
    if not text and payload.emailId:
        text = fetch_email_text_from_db(payload.emailId) # pseudo-function if not text:
        raise HTTPException(status_code=400, detail="No email text provided") # Call OpenAI API
    for summarization
    prompt = f"Summarize the following email in a few sentences:\n\n{text}\n\n"
    try:
        response = await openai.Completion.acreate(
            model="text-davinci-003",
            prompt=prompt,
            max_tokens=150,
```

```

        temperature=0.7
    )
    summary = response.choices[0].text.strip()

    14
    return {"summary": summary}
except Exception as e:
    print("OpenAI API error:", e)
    raise HTTPException(status_code=500, detail="AI summarization failed")

@app.post("/smart-replies")
async def generate_smart_replies(payload: EmailText, token: dict = Depends(verify_token)):
    """Generate smart reply suggestions for an email."""
    text = payload.text or fetch_email_text_from_db(payload.emailId) if not text:
        raise HTTPException(status_code=400, detail="No email text provided")
    prompt = f"Generate 3 brief, professional replies to the following email: \n\n\"{text}\n\n\nReplies:"
    try:
        response = openai.Completion.create(
            model="text-davinci-003",
            prompt=prompt,
            max_tokens=100,
            n=3, # ask for 3 completions
            stop=None,
            temperature=0.8
        )
        replies = [choice.text.strip() for choice in response.choices]
        return {"replies": replies}
    except Exception as e:
        print("Error generating replies:", e)
        raise HTTPException(status_code=500, detail="AI reply generation failed")

# Additional endpoints could be defined similarly (e.g., /embed, /analyze-tone)

```

In the above code, we use OpenAI's completion API for simplicity. In a real implementation, one might fine tune the prompts or use newer models (like gpt-4 via the ChatCompletion API) for better results. The `verify_token` dependency (not shown) would decode and validate the JWT, ensuring the request is ^{3 4} authorized – typically checking signature against Cognito's JWKS and expiry . For

embedding, if using OpenAI:

```

@app.post("/embed")
async def embed_text(payload: EmailText, token: dict = Depends(verify_token)):
    text = payload.text or fetch_email_text_from_db(payload.emailId) if not text:
        raise HTTPException(status_code=400, detail="No text to embed")
    response = openai.Embedding.create(model="text-embedding-ada-002",
        input=text[:1000]) # truncate to max length

```

```
vector = response['data'][0]['embedding']
# Optionally, store the embedding in OpenSearch or a vector DB here return {"embedding":
vector}
```

We might store these embeddings in **Amazon OpenSearch** (which supports vector search in recent versions) or another vector store, enabling semantic search queries like "find emails about X".

The AI Service, being stateless (just processing requests), scales easily on Kubernetes. We include a Dockerfile (using tiangolo/uvicorn-gunicorn-fastapi as base or similar) and configure the deployment with CPU/GPU resources as needed (if heavy ML, perhaps attach a GPU node group, but with OpenAI API usage, it's not needed).

4. Encryption Service (Signal Protocol & Vault)

Security is a core feature of Mangu Mail – the **Encryption Service** handles end-to-end encryption of email content using the **Signal protocol** and manages encryption keys via **HashiCorp Vault**. The aim is that if two Mangu Mail users communicate, their email content is E2E encrypted (so that even our servers cannot read it in plaintext, if we choose to implement zero-access encryption).

Signal Protocol Integration: We plan to use the Signal protocol (which uses a Double Ratchet algorithm for forward-secure messaging) for encrypting email bodies. Implementing Signal from scratch is complex, but we can leverage existing libraries. Since Signal's official implementation is in Java/Android and there are some community implementations in JavaScript (like @privacyresearch/libsignal or the old

libsignal-protocol-js), one approach is: - Use a **Service in Node.js** that incorporates a Signal library (for example, using the Rust libsignal-client via WASM or a prebuilt Node addon, or a pure JS variant if available). - Each user would have a **Signal identity (Identity Key Pair)** and one or more **Pre-Keys** for others to establish sessions with them. These keys must be generated and stored securely. - **Key Storage:** HashiCorp Vault is used to store these sensitive keys. Vault's KV secrets engine or the Transit engine can be used. For instance, when a user signs up, we generate a Signal identity key pair and store it in Vault (KV store under secret/data/signal/<userId>). Vault secures and controls access to these encryption keys – only the Encryption Service (with proper Vault token/role) can retrieve them. - When one user sends an email to another, the Encryption Service will retrieve the recipient's public identity and pre-key from Vault (or a database) to establish a session and encrypt the message with Signal's algorithm. The encrypted payload (ciphertext) is then stored or sent. - The recipient, when reading the email, will call the Encryption Service to decrypt the content (using their private keys stored in Vault) to plaintext.

Vault Integration: We configure Vault (likely running as a separate service in our cluster or using HashiCorp Cloud Vault) to manage secrets: - Vault is authenticated from the Encryption Service via a trusted method (could be a Kubernetes auth role or an AppRole). For simplicity, in development .env , we might use a Vault root token or AppRole ID/secret. - We store keys in Vault's KV store (which automatically encrypts at rest). Alternatively, Vault's **Transit secrets engine** can be used: the Transit engine can generate an encryption key and perform encryption/decryption as a service (the data never stored, just encrypted and returned) . Using Transit is simpler if we just want to offload encryption operations: e.g., enable a transit key named "mangu-mail", then Vault API can encrypt/decrypt data with that key. However, Transit is symmetric encryption – for E2EE with per-user keys, we might stick to managing keys manually (as in Signal). - We may use Vault's **PKI engine** or **Key Management** features if needed for managing other keys (DKIM keys could also be stored here instead of environment variable, for instance).

Endpoints: The Encryption Service might expose APIs like: - GET /public-key?userId=XYZ – to retrieve a user's public identity key (for another service or client to encrypt data to that user). - POST /encrypt – request body with toUserId , fromUserId , and plaintext (or reference to stored plaintext). The service then uses fromUserId's private and toUserId's public keys to do an X3DH/Signal encryption and returns the ciphertext (or stores it). - POST /decrypt – with userId and ciphertext , returns plaintext (if the userId's private key can decrypt it). - Possibly POST /register – to initialize keys for a new user (generate keys and store them). In practice, user registration triggers this behind the scenes.

However, since we rely on Cognito for auth, user registration will happen there. We might hook into Cognito triggers or simply have our backend call the Encryption Service to create keys when a new user is confirmed.

Signal Implementation: For demonstration, assume we find a JS library to perform the core steps (the actual Signal protocol involves one-time pre-keys, etc. but given time, we illustrate basic public-key encryption e.g. using X25519 keys): - We generate an **identity key pair** (Curve25519) for each user. - For each message, generate a symmetric session key via an ECDH between sender's private and recipient's public key (this is not the full double ratchet but a simple approach). - Encrypt the message with that symmetric key (e.g., using AES-GCM). - (A real Signal implementation would manage successive message keys and more, but that's beyond scope here.)

Code Snippet: Here's a conceptual snippet (not full Signal, but illustrative):

```
// encryption-service/src/index.ts (Express app for simplicity)
import express from 'express';
import bodyParser from 'body-parser';
import { encryptWithSignal, decryptWithSignal, generateUserKeys } from './signal';
import vault from './vault';

const app = express();
app.use(bodyParser.json());

// Middleware to verify JWT (Cognito)
app.use(async (req, res, next) => {
  const authHeader = req.headers.authorization;
  if (!authHeader) return res.status(401).send("Missing Authorization"); const token =
    authHeader.split(" ")[1];
  try {
    // verifyToken returns decoded payload if valid, throws if not
    req.user = await verifyCognitoToken(token);
    next();
  } catch (err) {
    return res.status(401).send("Invalid token");
  }
});
```



```
// Route to get public key of a user
app.get('/public-key/:userId', async (req, res) => {
  const targetUserId = req.params.userId;
  // Only allow accessing someone else's public key (or your own) try {
    const pubKey = await vault.read(`secret/data/signal/${targetUserId}/ publicKey`);
    if (!pubKey) return res.status(404).send("User key not found"); res.json({ userId:
      targetUserId, publicKey: pubKey });
  } catch(err) {
    res.status(500).send("Vault error");
  }
});

// Route to encrypt a plaintext message to a recipient
app.post('/encrypt', async (req, res) => {
  const fromUserId = req.user.sub; // assuming JWT sub is user id const { toUserId,
  plaintext } = req.body;
  if (!toUserId || !plaintext) {
    return res.status(400).send("toUserId and plaintext required"); }
  try {
    const senderKeys = await vault.read(`secret/data/signal/${fromUserId}`); const recipientPubKey =
    await vault.read(`secret/data/signal/${toUserId}/ publicKey`);
    if (!senderKeys || !recipientPubKey) {
      return res.status(404).send("Keys not found for encryption"); }
    const ciphertext = encryptWithSignal(plaintext, senderKeys.privateKey, recipientPubKey);
    res.json({ ciphertext });
  } catch (err) {
    console.error("Encryption error:", err);
    res.status(500).send("Encryption failed");
  }
});
```

```
// Route to decrypt a ciphertext for the authenticated user app.post('/decrypt',
async (req, res) => {
  const toUserId = req.user.sub;
  const { fromUserId, ciphertext } = req.body;
  if (!ciphertext || !fromUserId) {
    return res.status(400).send("fromUserId and ciphertext required"); }
  try {
    const recipientKeys = await vault.read(`secret/data/signal/${toUserId}`); const senderPubKey =
    await vault.read(`secret/data/signal/${fromUserId}/ publicKey`);
```

```
if (!recipientKeys || !senderPubKey) {
  return res.status(404).send("Keys not found for decryption");
```

```

    }
    const plaintext = decryptWithSignal(ciphertext, recipientKeys.privateKey, senderPubKey);
    res.json({ plaintext });
  } catch (err) {
    console.error("Decryption error:", err);
    res.status(500).send("Decryption failed");
  }
});

// (Possibly an internal route or function to generate and store keys on user registration)
async function initUser(userId: string) {
  const { publicKey, privateKey } = generateUserKeys();
  await vault.write(`secret/data/signal/${userId}`, { publicKey, privateKey }); }

// Start the server
app.listen(process.env.PORT || 7000, () => console.log("Encryption Service running"));

```

In this pseudo-code: - vault.read and vault.write represent calls to Vault's API (e.g., using Vault's Node SDK or simply REST calls with the service's token). - The encryption uses placeholder encryptWithSignal / decryptWithSignal which would implement the actual cryptography (perhaps using tweetnacl or Node crypto for an ECDH + symmetric encrypt). - verifyCognitoToken is similar to ³ earlier discussion – verifying the JWT signature and claims (could use aws-jwt-verify library) . - In practice, for multi-device support and true Signal protocol, more complexity is needed (session management, etc.), but this covers basic secure messaging.

Using Vault's Transit Engine: An alternative design is to use Vault's Transit engine as a simpler encryption service. For example, enable transit and name a key "mangu-mail-key". The Encryption Service could then call Vault Transit to encrypt/decrypt data without handling keys directly: - vault.encrypt('mangu-mail key', plaintext) returns ciphertext (which Vault produces using its managed key). -

vault.decrypt('mangu-mail-key', ciphertext) returns plaintext. However, transit uses a single key (or multiple named keys) – it's great for encrypting data at rest, but not a true end-to-end between distinct users. We want per-user keys, so we stick to managing those.

⁷ HashiCorp Vault ensures that encryption keys are **securely stored and access is controlled** . Only the Encryption Service has the rights to use/read them, preventing other services from accidentally exposing sensitive keys.

5. Additional Services: Marketplace, Calendar, Achievements, etc.

Phase 2 also mentions optional services for new features: - **Marketplace Service:** A microservice to support the "email stakes marketplace". While not fully specified, we assume users can trade or stake something

related to emails (perhaps like selling priority in someone's inbox, or trading unused inbox capacity, etc.). For now, we scaffold a Marketplace service that could list "offers" and handle transactions: - Tech stack: Node.js (Express) or Python (FastAPI). Could integrate with payment APIs if real transactions. - It might have endpoints like GET /offers , POST /offers (list a new offer), POST /buy etc. - Uses a database (could reuse Scylla or a dedicated table) to store marketplace listings and user balances. Phase 2's frontend shows a balance (userSlice includes balance: 542.75), so the Marketplace service might manage that. - Possibly

integrates with the Achievements (e.g., an achievement for first trade) or Social (posting a new listing). - For brevity, we include a placeholder service with basic structure and notes in README.

- **Calendar Service:** Manages calendar events and scheduling. Mangu Mail's calendar might extract event invites from emails (iCal attachments) and allow the user to see events in a calendar view:
- Endpoints like GET /events , POST /events to create personal events, etc. •
It could also subscribe to emails.received events to auto-create events when an email with a calendar invite arrives (e.g., if an email has an .ics attachment, the Email Processor could detect that and send an event that Calendar service uses to add to user's calendar).
- Uses ScyllaDB (or another DB) to store events. Could also integrate with external calendars (Google Calendar API) but likely out-of-scope.
- The front end has a CalendarPage and an example event in the state (with stakeValue possibly indicating some marketplace stake for meetings).
- We implement a simple service that stores events per user (with fields like title, time, participants).
- Could use a library like date-fns on front end for formatting dates (already included).
- **Achievements Service:** Tracks user achievements, streaks, and badges. This could be a rules engine or simple counters:
- Achievements might include "Inbox Zero for X days" or "Sent Y quick replies in under 5 minutes" etc.
- The service could consume Kafka events (e.g., when an email is marked read, increment a counter; when user replies within 5 min of receiving, trigger an achievement).
- Alternatively, the logic can partly reside in the frontend (the userSlice in Redux has some achievement data pre-filled for demo). But a backend service would ensure persistence and cross device consistency.
- We design it to subscribe to events like emails.received , emails.sent , etc., and update a database of achievements. For instance, maintain a counter of consecutive days inbox zero (if each night it checks).
- Endpoints: GET /achievements to list achievements for the user, POST /achievements/ •
unlock if some manual trigger, etc.
- **Social Service:** Not explicitly asked, but the front end has social features (social feed, connections, insights). This might be integrated within Achievements or separate:
- It could fetch data like a company's org chart or integrate with LinkedIn for "insights" about contacts (as the socialSlice example shows LinkedIn and Twitter info for a person).
-

Out of scope to fully implement, but we note it could call external APIs or maintain a social graph of contacts who use Mangu.

20

- **Workflow Service:** The “Workflows” feature (Kanban board) likely implies an automation or project management aspect. Possibly integrating with Temporal (or could be just a UI feature).
- A Workflows service could allow users to define email automation rules (like “if email from X, do Y”). This could be implemented via Temporal or a simple rules engine (like AWS Step Functions or a Node rule evaluator).
- Given the mention of **Temporal.io** in the architecture, we highlight using Temporal to orchestrate user-defined workflows or internal processes:
 - We deploy a **Temporal Worker** (as workflows-service). We define workflows in TypeScript, e.g., a FollowUpWorkflow that waits 3 days after sending an email and then sends a reminder if no reply, etc. This can correspond to user-set rules. Temporal server can be run as a service in K8s (or use Temporal Cloud). In our infrastructure,
 - we might deploy the Temporal server in the EKS cluster (with Cassandra or MySQL persistence, but to simplify, maybe use Temporal’s in-memory or a small PostgreSQL). Our Workflows service registers with the Temporal server and executes workflows and
 - activities as needed. For example, an activity might send an email (so it could call our Email Processor or send directly). This kind of integration ensures reliable execution of delayed or multi-step tasks (with retries,
 - etc.) .
 - We incorporate a basic example: a workflow that, when triggered by an event (like a certain email received), will schedule sending another email via an activity. This is advanced functionality and optional.

In summary, these optional services are scaffolded similarly to others: Express or FastAPI with their own DB tables and Kafka event handling. They all rely on Cognito JWT for auth and run in K8s.

Cross-Cutting Concerns and Integration

User Authentication & Authorization (AWS Cognito)

AWS Cognito is central to user auth in Phase 2. We have set up a **Cognito User Pool** for ManguMail. All user sign-ups and sign-ins occur through this managed service, so we do **not** store user passwords or handle custom auth tokens. Cognito provides: - **JWT Tokens:** After login, the frontend receives an **ID token** (and access/refresh tokens). This ID token is a JWT signed by Cognito (RSA-256) and includes user’s identity (e.g., sub claim as user ID, and other attributes). - We configure each backend service to **verify these JWTs** on each request. In Node.js services, we use AWS’s recommended `aws-jwt-verify` library to ³ validate token signature and claims . This involves caching Cognito’s public keys (from the JWKS URL) and verifying the token’s signature, issuer, audience, and expiration for each request. - Example middleware

(Node/Express):

```
import { CognitoJwtVerifier } from "aws-jwt-verify";
const verifier = CognitoJwtVerifier.create({
  userPoolId: process.env.COGNITO_POOL_ID!,
  tokenUse: "id", // or "access"
  clientId: process.env.COGNITO_APP_CLIENT_ID!
});
```

21

```
async function authMiddleware(req, res, next) {
  const authHeader = req.headers.authorization;
  if (!authHeader) return res.status(401).send("No token");
  const token = authHeader.split(" ")[1];
  try {
    const payload = await verifier.verify(token);
    req.user = payload; // attach decoded token payload for use
    next();
  } catch (err) {
    console.error("JWT verification failed:", err);
    return res.status(401).send("Unauthorized");
  }
}
// Use authMiddleware on protected routes
app.use("/api", authMiddleware);
```

This ensures only valid tokens from our Cognito user pool are accepted. (In FastAPI, we do similarly using python-jose or aws-jwt-verify in Python or a custom dependency check.)

- **No Custom Auth Service:** We removed Phase 1's /api/auth/login and /api/auth/register endpoints. Instead, the frontend will use Cognito's hosted UI or Amplify as described. There is no need for the backend to generate JWTs – it simply trusts Cognito's tokens.
- **Roles and Permissions:** If needed, we could define user roles or use Cognito Groups/Claims to authorize certain actions. For example, an "admin" group claim could allow access to certain admin APIs. Each service can inspect token claims if needed.

Database and Storage Integration

Multiple storage systems are used, each for specific purposes:

- **ScyllaDB (NoSQL Database):** ScyllaDB (a Cassandra-compatible database) is chosen for high throughput storage of emails and related data. We use Scylla to store:
- Emails (each email as a row, partitioned by user mailbox, for example).

- User profiles and settings (though some basic profile info might also live in Cognito attributes).
- Calendar events, if not too relational (Scylla can handle time-series data well).
- Possibly social feed posts or other semi-structured data.

Each service that needs DB access either has its own keyspace or shares a keyspace with distinct tables. For instance, an emails table for Email Processor/Sync, a calendar_events table for Calendar, etc. We use a Scylla client driver (DataStax Cassandra driver for Node, or cassandra-driver in Python) to interact.

Example table design for emails:

22

```
CREATE TABLE mangu_mail.emails (
  user_id text,
  email_id text,
  folder text, // "inbox", "sent", etc.
  subject text,
  from text,
  to list<text>,
  body text,
  body_html text,
  flags map<text, boolean>, // e.g., 'unread': true
  received_at timestamp,
  PRIMARY KEY ((user_id, folder), received_at, email_id)
);
```

This allows querying a user's inbox sorted by date, for example.

- **Amazon OpenSearch (Search Engine):** We set up an OpenSearch cluster (managed service) to index email content for full-text search and analytics. The AI service might also index vector embeddings here. The Email Processor or Sync service will send data to OpenSearch whenever a new email comes in or an email is updated:
- Use OpenSearch Index e.g. emails_index where each document contains email text, subject, sender, etc., plus user_id. This allows queries like "find emails containing 'project update' from Bob".
- We use the OpenSearch JS client (@opensearch-project/opensearch for Node) to index documents asynchronously. For example, after saving an email in DB, call
client.index({ index: 'emails_index', id: email_id, body: { user_id, subject, text, ... } }) .
- OpenSearch also can store vector embeddings and perform kNN search, which we could use for finding similar emails (using the embedding from AI service).

The front end could query a search API endpoint that our backend implements by querying

- OpenSearch.
- **Redis (ElastiCache):** Redis serves as a fast in-memory cache and messaging broker for certain features:
- **Cache:** Could cache frequent queries or token verifications. For instance, caching Cognito JWKS or user profile data.
- Could also be used for pub/sub for real-time features (though we have Kafka, but Redis Pub/Sub or Streams could be used for low-latency updates like notifying the frontend of a new email in realtime via WebSocket).
- If implementing real-time chat (maybe the “channels” feature in social, where users chat in real time), Redis could back a WebSocket gateway for publishing messages.
- Temporal can use Redis (but Temporal by default doesn’t, it uses its own server and DB).
- We deploy Redis via AWS ElastiCache (likely as a cluster in the VPC). The services connect using environment config for Redis endpoint.

23

- **Amazon S3 (Object Storage):** S3 is used for storing binary large objects:
- Email **attachments** are stored in S3 instead of the DB. When an email with attachments is processed, the Email Processor will upload attachments to an S3 bucket (e.g., mangumail attachments) and store only references (URLs or keys) in the email record. This offloads large data. • **Voice note recordings** (from the voice recorder feature) can be stored in S3 as well. The front end might upload the audio blob to S3 via a presigned URL and then include the link in the email or user’s drive.
- We also can host static assets (though primarily the web app is deployed via Amplify, user uploaded content still goes to S3).
- A separate S3 bucket might be used for front-end hosting if not Amplify (Amplify itself uses an S3 behind the scenes).
- The AWS_ACCESS_KEY_ID and AWS_SECRET_ACCESS_KEY in environment configs (as seen in Phase 1 .env.example) would allow the backend to interact with S3 (though if running in AWS, it’s better to use IAM roles for service accounts on EKS).
- Example usage: in Node, using @aws-sdk/client-s3 to put object:

```
import { S3Client, PutObjectCommand } from "@aws-sdk/client-s3"; const s3 = new
S3Client({ region: process.env.AWS_REGION }); const bucket =
```

```

process.env.S3_BUCKET_NAME;
// ... in the email processor when handling attachment: const command =
new PutObjectCommand({
  Bucket: bucket,
  Key: `${userId}/${emailId}/${filename}`,
  Body: attachmentContent,
  ContentType: attachment.mimeType
});
await s3.send(command);

```

We then store the S3 key or URL in the email record for retrieval.

All these storage layers ensure the application is scalable and data is persisted appropriately. README files for each service detail how to set up connectivity (e.g., Scylla contact points, keyspace names, S3 bucket names, etc.).

Event-Driven Communication (Kafka and Temporal)

We use **Apache Kafka** (via AWS MSK) as the backbone for inter-service communication and decoupling. By producing events for significant actions, we enable flexible consumption and extension: - The Email Sync and Processor services produce events like `emails.new`, `emails.received`, `emails.sent`. - The AI service could consume `emails.new` events to automatically generate summaries for new emails in the background (so when a user opens an email, a summary might already be ready). This could be done by a small Kafka consumer in the AI service, or via a Temporal workflow. - The Achievements service consumes events (e.g., `emails.sent`) to update streak counts (like “quickReply” streak if email was replied quickly). - The Calendar service might consume `emails.received` to look for meeting invites. - The Social service could consume events to generate a feed (e.g., “User X sent an email to Y” if that’s something to share, or

24

milestone events). - Kafka topics might also handle user-related events (if not using Cognito triggers) like `user.registered` which could prompt the Encryption service to generate keys, etc.

All services use a common Kafka client and share schemas for the messages. For maintainability, we could use a schema registry (Confluent Schema Registry) or simply ensure consistent JSON structures.

Temporal Workflows: We introduced **Temporal** to manage complex asynchronous workflows reliably: - We deploy Temporal server on EKS (with persistence to a keyspace in Scylla or a separate database). - The **Workflows service** runs Temporal workers with our workflow code. - Example usage: when an email arrives, instead of just a one-off event, we might start a **workflow** like `ProcessNewEmailWorkflow`: - Step 1 (activity): Run spam filter & classification (though we did that synchronously in Email Processor, this could be decoupled). - Step 2 (activity): Generate an AI summary (call AI service or if simpler, do it within an activity function by calling OpenAI API). - Step 3 (activity): Perhaps auto-categorize the email (e.g., determine if it’s important or a newsletter using AI). - These activities can be retried automatically on failure, and the workflow can wait for all to complete. Temporal ensures if the service crashes in the middle, it can continue where left off. - Another workflow example: **SendEmailWithRetryWorkflow** that attempts to send via SMTP, and if it fails (e.g., external server down), retries with backoff up to 3 times (Temporal has built-in retry policies). - **User-defined workflows:** We could allow users to create rules like “if an email from boss

arrives, create a task in my Kanban and send a Slack message”. This could be implemented by having pre defined workflow templates that integrate with external APIs. The “Workflows” feature might eventually allow custom automations, which Temporal is well-suited for.

Temporal works alongside Kafka: Kafka is event streaming, Temporal is workflow orchestration. They can be used together (for instance, Kafka event triggers a Temporal workflow). The inclusion of Temporal ensures **reliability and stateful orchestration** beyond simple pub-sub.

We supply a basic **Temporal workflow code** in the workflows-service . For example, a snippet (TypeScript):

```
// workflows-service/src/workflows/newEmailWorkflow.ts
import { proxyActivities } from '@temporalio/workflow';
import type * as Activities from '../activities/newEmailActivities';

const { summarizeEmail, classifyEmail } = proxyActivities<typeof Activities>({ startToCloseTimeout: '30s'
});

export async function processNewEmailWorkflow(email: { id: string, content: string }) {
  // Concurrently run AI summarization and classification
  const [summary, category] = await Promise.all([
    summarizeEmail(email.id, email.content),
    classifyEmail(email.id, email.content)
  ]);
  // You could then store the results (this could be another activity) 25

  // e.g., await saveMetadata(email.id, { summary, category });
}
```

And in activities/newEmailActivities.ts :

```
import axios from 'axios';
export async function summarizeEmail(emailId: string, content: string): Promise<string> {
  // call the AI service to get summary
  const res = await axios.post(process.env.AI_SERVICE_URL + '/summarize', { text: content }, {
    headers: { 'Authorization': `Bearer ${process.env.SERVICE_JWT}` } // might use a service auth token
  });
  return res.data.summary;
}
export async function classifyEmail(emailId: string, content: string): Promise<string> {
  // simple classification, e.g., check for keywords or ML model
  if(content.match(/meeting|schedule|calendar/i)) return 'calendar';
  if(content.match(/unsubscribe/i)) return 'newsletter';
  // ...
  return 'general';
}
```

This example shows how a workflow can orchestrate multiple steps (activities) when a new email arrives. In practice, the Email Processor after saving an email could signal the Temporal workflow trigger via the Temporal client.

ArgoCD ensures that all these microservices and Temporal itself are deployed and configured as per our GitOps manifests, keeping the system in sync with our repo state.

AWS Infrastructure & Deployment

Phase 2 requires full infrastructure provisioning on AWS using Terraform, and deployment via Kubernetes + ArgoCD. We provide Terraform scripts and K8s manifests as part of the codebase in the `infra/` directory.

Infrastructure Provisioning (Terraform)

Our Terraform configuration defines all necessary AWS resources. We organize it by components (for readability, possibly using Terraform modules):

26

Key resources to provision:

- **VPC & Networking:** A new VPC with subnets (public for ALB, private for EKS nodes, etc.), internet gateway, NAT gateways for private subnets, and appropriate security groups.
- **EKS Cluster:** We create an Amazon EKS cluster to run our microservices. Terraform will provision the EKS control plane, node groups (perhaps one node group for general services and one with larger instances if needed for heavy services). We also set up the AWS authenticator so that ArgoCD and CI/ CD can access it.
- **IAM Roles:**
 - An IAM role for the EKS nodes (with policies to allow access to S3, MSK, etc. as needed).
 - IAM roles for service accounts (IRSA) e.g., to let pods talk to S3 or Secrets Manager without embedding keys.
 - Cognito-specific roles if needed (e.g., for Cognito Lambda triggers if used).
- **Amazon Cognito:** We define a Cognito User Pool with required attributes (email, name, etc.), an App Client for our frontend (with callback URLs if using hosted UI), and possibly a Domain for the hosted UI. We allow username/password signup and possibly federation (not explicitly requested, but Amplify could easily integrate social logins if desired).
- **Amazon MSK (Kafka):** We create a Managed Streaming for Kafka cluster. For simplicity, a small

cluster (3 brokers) in the private subnets. We also create Kafka topics via Terraform or after deployment (could be done via code on startup as well). MSK is configured to allow our services (in the VPC) to connect. We might need a MSK security group and an AWS Secrets Manager secret for Kafka credentials if using SASL, but internal access within VPC might be fine with plaintext or TLS without auth.

- **ElastiCache (Redis):** We provision a Redis cluster (likely cluster mode disabled, single primary with replicas or a small cluster). Placed in private subnets. Security group allows EKS nodes to connect.

- **Amazon OpenSearch Service:** We create an OpenSearch domain for our email indexing. This will be within the VPC, with an endpoint that our services can reach. We set appropriate access policy (access from VPC and perhaps a cognito master user for OpenSearch if needed for management).

S3 Buckets:

- One for email attachments and user files (e.g., mangu-mail-uploads). We enable versioning and possibly encryption (SSE). We might also separate one for voice notes or just use one bucket with prefixes.
- (If not using Amplify Console) one for hosting the frontend (with static website hosting or CloudFront). But since we plan to use Amplify for ease, we might skip manual S3 for hosting.
- **AWS Amplify:** Terraform can configure an Amplify Application connected to our repository. Alternatively, we might not manage Amplify via Terraform but rather via console. But to be thorough, we can include an Amplify App resource and deployment.

Other AWS Services:

- If using Amazon SES for sending email (as fallback or primary), we would verify a domain in SES and possibly use it. This can be set up manually; Terraform support exists for domain identity and DKIM in SES. Given we use our own SMTP, SES is optional.
- Amazon CloudWatch Logs: EKS will send logs; Terraform can create log groups if needed or we rely on EKS default.
- Amazon Certificate Manager (if we want TLS for a custom domain for the frontend or API, we'd request a certificate).
- AWS Route53: if managing DNS for our domain, to route mail (MX records), and for any user-facing domains.

27

We also include **Terraform for Temporal** if self-hosting: - Deploy a Cassandra or PostgreSQL for Temporal's persistence (Temporal can use MySQL/Postgres or Cassandra. We already have Scylla (Cassandra compatible) which might be used. But mixing app data with Temporal data might not be ideal, possibly a separate keyspace). - Deploy Temporal server (could be a Helm release). This might be outside Terraform's scope or done via K8s manifests.

Example Terraform snippet: (For brevity, one resource example – EKS cluster creation)

```
# infra/terraform/eks.tf
resource "aws_eks_cluster" "mangu" {
```

```

name = "mangu-mail-cluster"
version = "1.27" # EKS Kubernetes version
role_arn = aws_iam_role.eks_cluster_role.arn
vpc_config {
    subnet_ids = [aws_subnet.private1.id, aws_subnet.private2.id,
aws_subnet.private3.id]
    endpoint_private_access = true
    endpoint_public_access = true
}
}

resource "aws_eks_node_group" "mangu_default" {
    cluster_name = aws_eks_cluster.mangu.name
    node_group_name = "mangu-default-nodes"
    node_role_arn = aws_iam_role.eks_node_role.arn
    subnets = [aws_subnet.private1.id, aws_subnet.private2.id, aws_subnet.private3.id]
    scaling_config {
        desired_size = 3
        min_size = 3
        max_size = 6
    }
    instance_types = ["t3.medium"]
}

```

We would similarly have terraform config files for Cognito (using `aws_cognito_user_pool` and `aws_cognito_user_pool_client`), for MSK (`aws_msk_cluster`), for S3 (`aws_s3_bucket`), etc. All secrets (like Cognito client secret if any, or DB passwords) can be stored in AWS Secrets Manager or SSM Parameter Store and referenced.

We ensure outputs from Terraform (like cluster kubeconfig, Cognito Pool ID, etc.) are captured for use in deployment.

Kubernetes Manifests & Deployment (Helm/ArgoCD)

All microservices are containerized with **Dockerfiles**. We provide a Dockerfile in each service directory: - Base images: For Node services, use an official Node.js 18-alpine base, installing only production deps. For Python FastAPI, use python:3.10-slim with pip install of requirements (including `uvicorn[standard]`). - Each Dockerfile copies the source, installs dependencies, and sets the startup command (e.g., CMD ["node", "src/server.js"] or `uvicorn app.main:app --host 0.0.0.0 --port 80`). - We ensure to `.dockerignore` node_modules and other unnecessary files for smaller images.

Kubernetes Manifests: We include either raw YAML files or Helm charts for each service's Deployment, Service, ConfigMap/Secret, etc. Key aspects: - Each service gets a Deployment with a certain number of replicas (e.g., 2 or 3 for stateless services behind a load balancer). Likely use Deployment for stateless APIs

and StatefulSet if any service needed stable identity (probably not, except maybe Temporal or Kafka clients). - Containers have env vars sourced from Kubernetes Secrets/ConfigMaps (which are populated from our .env values or AWS Secret Manager via external secrets). - We mount necessary volumes if any (for example, if we needed a volume for Apache James maildir, but since we handle in code, not needed; or if SpamAssassin required rules volume). - Services that need to talk to each other can use internal DNS (e.g., http://ai-service:8000 from another pod). - **Ingress:** We likely set up an ALB Ingress or NGINX Ingress. Suppose we have a single API gateway service that clients talk to, then only that service needs an Ingress (which could be ALB with Cognito authentication if desired, or just a public endpoint that expects JWTs). Alternatively, expose each service under a subpath or subdomain: - e.g.,

api.mangumail.com/ai/* -> routes to ai-service, etc. Using an NGINX ingress with path routing. - But easier: We maintain the Phase 1 approach of a unified /api/* endpoint by running an API Gateway service or using Kubernetes Ingress with rewrite. For now, consider deploying an NGINX ingress and create ingress rules. -

ArgoCD: We treat our infra/k8s-manifests/ as the source for ArgoCD. We include an ArgoCD Application manifest (which ArgoCD itself would use) pointing to our repo and paths: - For example, an Application CR that points to the mangu-mail-app/infra/k8s-manifests directory and auto syncs it to the EKS cluster. - We might have multiple ArgoCD apps (one per service or one for all). - In our deliverables, we include infra/k8s-manifests/argocd/application.yaml like:

```
```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
 name: mangu-mail
 namespace: argocd
spec:
 project: default
 source:
 repoURL: https://github.com/user/mangu-mail-app.git
 targetRevision: HEAD
 path: infra/k8s-manifests
 destination:
 server: https://kubernetes.default.svc
 namespace: mangu-mail
```

29

```
syncPolicy:
 automated:
 prune: true
 selfHeal: true
```
```

This will cause ArgoCD to deploy all manifests in that path automatically. To illustrate, here's a

Deployment YAML for the AI service as an example:

```
# infra/k8s-manifests/ai-service-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
```

```

name: ai-service
labels:
  app: ai-service
spec:
  replicas: 2
  selector:
    matchLabels:
      app: ai-service
  template:
    metadata:
      labels:
        app: ai-service
    spec:
      containers:
        - name: ai-service
          image: myregistry/mangu-mail-ai:latest # assume images are built & pushed
          ports:
            - containerPort: 80
          env:
            - name: OPENAI_API_KEY
              valueFrom:
                secretKeyRef:
                  name: ai-service-secrets
                  key: OPENAI_API_KEY
            - name: AWS_REGION
              value: us-east-1
            - name: COGNITO_POOL_ID
              valueFrom:
                secretKeyRef:
                  name: global-secrets
                  key: COGNITO_POOL_ID
            - name: COGNITO_APP_CLIENT_ID

```

30

```

valueFrom:
  secretKeyRef:
    name: global-secrets
    key: COGNITO_APP_CLIENT_ID
# Other env vars like endpoints for DB, etc., if needed
# Optionally, serviceAccountName if using IRSA for AWS access

```

And a corresponding Service:

```

apiVersion: v1
kind: Service
metadata:
  name: ai-service

```

```
spec:
  selector:
    app: ai-service
  ports:
    - port: 80
      targetPort: 80
      protocol: TCP
      name: http
  type: ClusterIP
```

We'd have similar manifests for each microservice. For the API gateway (if implemented), we might create an **Ingress** resource:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: api-gateway-ingress
  annotations:
    kubernetes.io/ingress.class: alb
    alb.ingress.kubernetes.io/listen-ports: '[{"HTTP": 80}]'
    alb.ingress.kubernetes.io/scheme: internet-facing
    # (If we want Cognito OAuth2 authentication at ALB level, we could use
    alb.ingress.kubernetes.io/auth-type: cognito and related settings for User Pool) spec:
  rules:
    - host: api.mangumail.com
      http:
        paths:
          - path: /api/ai/*
            pathType: Prefix
            backend:
              service:
```

31

```
      name: ai-service
      port:
        number: 80
    - path: /api/encryption/*
      pathType: Prefix
      backend:
        service:
          name: encryption-service
          port:
            number: 80
    - path: /api/*
      pathType: Prefix
      backend:
        service:
          name: gateway-service
```

port:

number: 5000 # if we have a Node gateway on 5000

In this setup, either we direct certain paths to specific services, or we funnel everything to a gateway that then routes internally (the gateway approach might simplify front-end configuration, as it only calls one base URL). The question didn't explicitly mention a gateway, but Phase 1 had a single Express backend handling multiple endpoints. In microservice world, an API Gateway (could be as simple as an Nginx reverse proxy or a dedicated service like AWS API Gateway) would be ideal. Since we have Kubernetes, using an Ingress with path routing as above can serve as the gateway.

Terraform & K8s Integration: We output the EKS cluster kubeconfig and cluster endpoint, which could be used to configure ArgoCD's cluster access. We might also deploy ArgoCD itself on the cluster (via a manifest or Helm in Terraform). If doing GitOps properly, ArgoCD is installed and then ArgoCD pulls the rest of manifests. For completeness, we might include instructions to install ArgoCD (manifest or Helm chart).

Environment Configuration and Secrets

Each service's `.env.example` file enumerates the config it needs. For instance: - email-sync service/.env.example :

```
JMAP_BASE_URL=https://jmap.fastmail.com/api/
JMAP_ACCOUNT_ID=user@example.com
JMAP_TOKEN=your_jmap_api_token
KAFKA_BROKER=b-1.mskcluster.amazonaws.com:9092
SCYLLA_CONTACT_POINTS=scylla-cluster.mangu.svc.cluster.local:9042 -
```

email-processor-service/.env.example :

```
SMTP_PORT=2525
DKIM_DOMAIN=mangumail.com
```

32

```
DKIM_KEY_SELECTOR=mangu
DKIM_PRIVATE_KEY_PATH=./dkim-private.pem # or the key inline as shown earlier
SPAMASSASSIN_HOST=spamassassin
SPAMASSASSIN_PORT=783
KAFKA_BROKER=b-1.mskcluster.amazonaws.com:9092
SCYLLA_CONTACT_POINTS=...
S3_BUCKET_NAME=mangu-attachments
AWS_REGION=us-east-1
```

- ai-service/.env.example :

```
OPENAI_API_KEY=your_openai_api_key
HUGGINGFACE_API_TOKEN=your_hf_token (if needed)
```


- encryption-service/.env.example :

```
VAULT_ADDR=http://vault.mangu.svc.cluster.local:8200
VAULT_TOKEN=<devtoken> # in prod, use AppRole or K8s auth, so maybe VAULT_ROLE_ID
instead
COGNITO_POOL_ID=us-east-1_xxx
COGNITO_APP_CLIENT_ID=xxx
```

- And similarly for others (marketplace might need none aside from DB; calendar just DB).

The .env.example files guide developers to set their local dev environment. In AWS/K8s, these values are set via Secrets and ConfigMaps, not via a file.

Docker Compose (for Dev): Although not explicitly asked, a nice addition is a docker-compose file to run all services locally (with maybe local Kafka, Scylla, etc., perhaps using images). We include instructions in READMEs for running in dev mode (possibly using a simpler setup or mocks if not running the full infra).

README Documentation

Each service's README.md will summarize: - The service's purpose and features. - How to run it (e.g., "yarn install, then yarn start" or using Docker). - The API endpoints it exposes (if any). - The environment variables required (pointing to .env.example). - Any special instructions (for example, Email Processor README might say "you need SpamAssassin running locally for full functionality" or how to generate DKIM keys; AI Service README might say "set your OpenAI API key in env"). - The Encryption Service README will likely have setup notes for Vault (like how to enable the transit engine or set up the Vault dev server). - We also have a top-level README (in Phase 1 we had one) which we will update to reflect the new architecture: - It can have a section for "Microservices Overview" listing each service and its port. - A section for "Deployment" that instructs how to use Terraform to provision AWS resources and then how to deploy to EKS (e.g., pushing Docker images and applying manifests or using ArgoCD). - Possibly a diagram (if this were a real code deliverable, a diagram would be great to include, but here we describe it in text).

33

Ensuring Build and Deployment

Finally, we ensure the codebase is production-grade: - Linting and formatting in each service (ESLint for Node, Flake8 for Python, etc.). - All APIs are versioned under /api path and use JSON. - We handle errors gracefully and use secure defaults (e.g., Helmet middleware in Express for security headers, CORS allowed origins configured if needed). - Each service is **stateless** so it can scale horizontally. Kafka and DB handle coordination. - The system is cloud-ready with 12-factor principles (config via env, etc.).

By following this design and providing all the code scaffolding and config, the Phase 2 Mangu Mail is a robust, scalable application. It can be built and deployed with minimal modifications: just set up the AWS infra via Terraform, build/push Docker images, and apply the Kubernetes manifests (or let ArgoCD do it). All services will register healthy and start communicating through Kafka and REST, and the frontend (once pointed to the correct API endpoint) will offer the rich experience as designed.

References

1. JMAP protocol overview – JMAP is a JSON-based, efficient protocol to synchronize mail, intended to replace IMAP/SMTP for modern clients¹. It allows querying, fetching, and sending mail in a single consolidated API.
2. AWS Cognito JWT verification – In Node.js applications, AWS provides the aws-jwt-verify library to validate Cognito JWT tokens, ensuring the token's signature and claims (issuer, audience,³ expiration) are valid. This is used in each microservice to authenticate requests.
Temporal workflows – Temporal.io allows defining durable workflows with built-in retries and scheduling. For example, it can automatically retry failed email sending tasks and even include delays (using sleep()⁹ in a workflow) without losing state, which greatly simplifies implementing complex email sequences.
4. HashiCorp Vault for key management – Vault securely stores and controls access to secrets and encryption keys⁵. We utilize Vault to store Signal protocol keys, ensuring that private keys remain encrypted at rest and are only accessible by the Encryption Service. This approach strengthens the security of end-to-end encryption in the system.

¹ ²

JMAP Mail Specification

<https://jmap.io/spec-mail.html>

tokens - Amazon Cognito

³ ⁴

Verifying JSON web

<https://docs.aws.amazon.com/cognito/latest/developerguide/amazon-cognito-user-pools-using-tokens-verifying-a-jwt.html>

HashiCorp Developer

⁵ ⁶ ⁷

Why use Vault | Vault |

<https://developer.hashicorp.com/vault/tutorials/get-started/why-use-vault>

Solution | Temporal

⁸ ⁹

Node.js Task Queue

<https://temporal.io/blog/using-temporal-as-a-node-task-queue> ³⁴